

Reviewer Pack — Minimal Rank of Representations Bounds Circuit Monotonicity

Entry 07dccb13ed54 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 23:57:59 UTC

Minimal Rank of Representations Bounds Circuit Monotonicity

Entry ID: 07dccb13ed54 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-27 23:57:59 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Schur-Weyl Duality) **Field B** (complexity object): Complexity Theory: Boolean Circuit Monotonicity

Statement:

[‘For each positive integer n , there exists a constant $c(n)$ such that the minimal rank of any irreducible representation ρ in the Schur-Weyl duality category corresponding to a monomial ideal with at most n variables is at least $c(n)$. Specifically, for all boolean functions $f: \{0,1\}^n \rightarrow \{0,1\}$, there exists an irreducible representation ρ such that the minimal rank of $\rho(f)$ is at least $c(n)$, where $c(n) = 2^{\lfloor n/2 \rfloor}$.’, ‘For a given n , this implies that the monotone complexity of any function f with at most n variables is bounded from below by $2^{\lfloor n/2 \rfloor}$ in terms of circuit depth.’, ‘This conjecture provides a lower bound on the size of monotone circuits for arbitrary boolean functions based on representation theory.’]

Rationale (proposer’s reasoning):

[‘The Schur-Weyl duality offers a powerful tool to study representations of Lie algebras and related algebraic structures, which can be applied to represent boolean functions. By leveraging this representation theory, we may uncover novel insights into the inherent complexity of monotone circuits.’, “Monotonicity in circuit complexity refers to the property that a function’s output is always non-decreasing as its inputs increase. Understanding the lower bounds for monotone circuits could lead to progress in proving lower bounds for general boolean functions.”, ‘Previous works have used representation theory to study the complexity of polynomial equations and matrix multiplication, suggesting potential for extension to circuit complexity.’]

Taxonomy category: GCT_DET_PERM (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 9b831db349247bfe

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

A conjecture states that for a boolean function f with at most n variables, there exists an irreducible representation $\rho(f)$ with minimal rank $\geq 2^{(n/2)}$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "Schur-Weyl duality" AND "Boolean circuit monotonicity" AND minimal rank" - "irreducible representation" AND "monomial ideal" AND "circuit monotonicity" - "representation theory" AND "complexity theory" AND "lower bound on circuit size"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤ 90 s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def schur_weyl_representation(f):
        n = len(f)
        if n == 1:
            return [[f[0]]]
        else:
            f1 = f[:2**(n-1)]
            f2 = f[2**(n-1):]
            rep1 = schur_weyl_representation(f1)
            rep2 = schur_weyl_representation(f2)
            result = []
            for r1 in rep1:
                for r2 in rep2:
                    new_rep = [r1[i] + r2[j] for i, j in zip(range(2**(n-1)), range(2**(n-1)))]
                    result.append(new_rep)
            return result

    def minimal_rank(rep):
```

```

    rank = 0
    for row in rep:
        if any(row):
            rank += 1
    return rank

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    f = generate_boolean_function(n)
    rep = schur_weyl_representation(f)
    rank = minimal_rank(rep)
    results.append(rank)

mean_rank = sum(results) / len(results)
conjecture_holds = all(rank >= 2**(n/2) for rank, n in zip(results, n_values))
counterexample = "" if conjecture_holds else "mapping_undefined"

return {
    "metric_name": "Minimal Rank",
    "metric_value": mean_rank,
    "instances_tested": len(n_values),
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 1 for i in range(5, 30)]

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_value = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction=1.0")
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_value} std=0.0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_fai

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

```

Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_55965f62.py", line 74, in <module>
    trial_result = run_trial(seed)
                    ^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_55965f62.py", line 52, in run_trial
    rep = schur_weyl_representation(f)
          ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_55965f62.py", line 31, in schur_weyl_re
    rep1 = schur_weyl_representation(f1)
          ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_55965f62.py", line 31, in schur_weyl_re
    rep1 = schur_weyl_representation(f1)
          ^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_55965f62.py", line 31, in schur_weyl_re
    rep1 = schur_weyl_representation(f1)
          ^^^^^^^^^^^^^^^^^^^^^^^^^
[Previous line repeated 995 more times]
RecursionError: maximum recursion depth exceeded

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means that the pre-registered support condition could not be unambiguously met. | next: Re-run the test with increased recursion limits or a different implementation to verify the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13860
2	preregistration	ollama_remote	glm4:latest	0	0	6020
3	novelty	ollama_remote	glm4:latest	0	0	4877
4	novelty	ollama_remote	glm4:latest	0	0	5677
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	39265
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8080
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7070
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8202
9	judge	ollama_remote	glm4:latest	0	0	27257

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 120307 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in [research/audit/07dccb13ed54.jsonl](#))

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/07dccb13ed54.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/07dccb13ed54.tar.gz` (if generated)